01.12.2025

7. WAP to implement doubly linked list with primitive operations

a) create doubly linked list

b) insert a new node to the left of the node.

c) delete a node based on specified value.

d) display the contents of list.

pseudocode:

```
struct node {
    struct node *prev, *next;
    int data
}

insertatfront

Void insertatfront(int data) {
    struct node *newnode (
    newnode -> data = data
    newnode -> prev = null
    newnode -> next = head
    if (head == null) head = tail = newnode
    else  head -> prev = newnode
          head = newnode
Void insertatend (int data) {
    struct node * newnode (
    newnode -> data = data
    newnode -> next = null
    newnode -> prev = tail
    if (tail = null) head = tail = newnode
    else tail -> next = newnode, tail = newnode

Void insertatpos (int data, pos)
    int i
    struct node *newnode
case 1 : if pos = 1
        insertatfront (data)
case 2 : for (i=1; i<pos-1 & temp != null; i++) temp = temp -> next
        if (temp == null || temp -> next = null)
            insertatend (data)
newnode -> next = temp -> next
newnode -> prev = temp
temp -> next -> prev = newnode
```

```
temp->next = newnode

void deleteatfront()
    struct node *temp
    if (head = null) printf(empty)
    temp = head.
    head = head->next
    if (head != null) head->prev = null
    else tail = null
    free(temp)

void deleteatend()
    struct node *temp
    if (tail = null) printf(empty)
    ~~tail~~ = temp = tail;
    tail = tail->prev
    if (tail != null) tail->next = null
    else head = null;
    free(temp).

void deletebyvalue(int value)
    struct node *temp = head
    if (head = null) print(empty)
    while( temp != null && temp->data != value)
        temp = temp->next
    if (temp == null) print(element not found)
    if (temp = head) deleteatfront()
    if (temp = tail) deleteatend()
    else
        temp->prev->next = temp->next
        temp->next->prev = temp->prev
        free(temp)

void display()
    struct node *temp = head
    while (temp != tail)
        printf("%d", temp->data)
        temp = temp->next
    return.
```

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *prev, *next; };
struct Node *head= NULL, *tail=NULL;
void createList(int n) {
        int i, data;
        struct Node *newNode;
        For(i=1 ; i<=n; i++) {
            printf("Enter data for node %d : ",i);
            scanf("%d", &data);
            newNode=(struct Node*)malloc(sizeof(struct Node));
            newNode->data=data;
            newNode->prev=newNode->next = NULL;
            IF(head==NULL) {
                head=tail= newNode;
            }else{
                tail->next=newNode;
                newNode->prev=tail;
                tail=newNode; }}};
void insertatfront(int data){
    struct Node *newNode=(struct Node*) malloc(sizeof(struct
                                                    Node));
    newNode->data =data;
    newNode->prev =NULL;
    newNode->next= head;
    IF(head==NULL)
        head=tail=newNode;
    else{
        head->prev=newNode;
        head=newNode;
        }
}
void insertatend(int data){
    struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
    newnode->data= data;
    newnode->prev = ~~NULL~~ tail;
```

```
    newnode->next = NULL;
    if (tail == NULL)
        head = tail = newnode;
    else {
        tail->next = newnode;
        tail = newnode;
    }
}
void insertatposition (int data, int pos) {
    int i;
    struct Node *newNode, * temp = head;
    if (pos == 1) {
        insertatfront(data);
        return;
    }
    newnode = (struct node*) malloc ( sizeof( struct Node));
    newNode -> data = data;
    For(i=1 ; i< pos-1 && temp != NULL ; i++)
        temp = temp->next;
    if (temp == NULL || temp->next == NULL) {
        insertatEnd (data);
        return;
    }
    newNode ->next = temp ->next;
    newNode->prev = temp;
    temp -> next -> prev = newNode;
    temp ->next = newNode;
}

Void deleteatfront() {
    struct Node *temp;
    if (head == NULL) {
        printf("List is empty \n");
        return;
    }
    temp = head;
    head = head -> next;
    if (head != NULL)
        head -> prev = NULL;
    else
        tail = NULL;
    free(temp);
}
```

```
void deleteatend( ){
    struct node *temp;
    if(tail==NULL){
        printf("list is empty\n");
        return;
    }
    temp=tail;
    tail=tail->prev;
    if(tail!=NULL)
        tail->next=NULL;
    else
        head=NULL;
    free(temp);
}
void deletebyvalue(int value){
    struct node *temp=head;
    if(head==NULL){
        printf("List is empty\n");
        return;
    }
    while(temp!=NULL && temp->data!=value)
        temp=temp->next;
    if(temp==head)
        deleteatfront();
    else if(temp==tail)
        deleteatend();
    else
        temp->prev->next=temp->next;
        temp->next->prev=temp->prev;
        free(temp);
    }
}
void display(){
    struct node *temp=head;
    printf("List:");
    while(temp->next!=NULL){
        printf("%d<->",temp->data);
        temp=temp->data;
    }
    printf("NULL\n");
}
```

```
int main(){
    int choice, n, data, pos, value;
    printf("Enter number of nodes to create: ");
    scanf("%d", &n);
    createlist(n);
    while(1){
        printf("\n1. Display\n");
        printf("2. Insert at Front\n");
        printf("3. Insert at End\n");
        printf("4. Insert at position\n");
        printf("5. Delete at Front\n");
        printf("6. Delete at End\n");
        printf("7. Delete by value.\n");
        printf("8. Exit\n");
        printf("Enter choice:");
        scanf("%d", &choice);
    switch(choice){
        case 1:
            display();
            break;
        case 2:
            printf("Enter data:");
            scanf("%d", &data);
            insertatfront(data);
            break;
        case 3:
            printf("Enter data:");
            scanf("%d", &data);
            insertatEnd(data);
            break;
        case 4:
            printf("Enter data and position:");
            scanf("%d%d", &data, &pos);
            insertatposition(data, pos);
            break;
        case 5:
            deleteatfront();
            break;
        case 6:
            deleteatEnd();
            break;
        case 7:
            printf("Enter value to delete:");
            scanf("%d", &value);
            deletebyvalue(value);
```

```
            break;
        case 8: exit(0);
        default: printf("Invalid choice !\n");
    }
  }
  return 0;
}
```

O/P

Enter number of nodes to create : 3

Enter data for node1 : 10

Enter data for node2 : 20

Enter data for node3 : 30

1. Display
2. Insert at front
3. Insert at end
4. Insert at position
5. Delete at front
6. Delete at end
7. Delete by value
8. Exit

Enter choice : 2

Enter data : 5

1. Display
2. Insert at front
3. Insert at end
4. Insert at position
5. Delete at front
6. Delete at end
7. Delete by value
8. Exit

Enter choice : 3

Enter data : 40

1. Display
2. Insert at front
3. Insert at end
4. Insert at position
5. Delete at front
6. Delete at end
7. Delete by value
8. Exit

Enter choice : 4

Enter data and position : 45 3

1. Display
2. Insert at front
3. Insert at end
4. Insert at position
5. Delete at front
6. Delete at end
7. Delete by value
8. Exit

Enter choice : 5

1. Display
2. Insert at front
3. Insert at end
4. Insert at position
5. Delete at front
6. Delete at end
7. Delete by value
8. Exit

Enter choice : 6

1. Display
2. Insert at front
3. Insert at end
4. Insert at position
5. Delete at front
6. Delete at end.

7. Delete by value

8. Exit

Enter choice: 7

Enter value to delete: 30

1. Display
2. Insert at front
3. Insert at end
4. Insert at position
5. Delete at front
6. Delete at end
7. Delete by value
8. Exit

Enter choice: 1

List: 10 ⟷ 45 ⟷ 20 ⟶ NULL

1. Display
2. Insert at front
3. Insert at end
4. Insert at position
5. Delete at front
6. Delete at end
7. Delete by value
8. Exit

Enter choice: 8

08.12.2025

Write a program a) to construct a binary search tree.
b) to traverse the tree using all the methods i.e in order, pre-order and post order.
c) to display the elements.

```
#include<stdio.h>
#include<stdlib.h>
struct Node {
    int data;
    struct Node *left, *right;
};
struct Node* createNode(int value){
    struct Node *newNode = (struct Node*) malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode; }
```